

# Numerical Methods: Comprehensive Code and Theory Reference

## Python Implementation Reference

### Contents

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Root-Finding Methods</b>                                  | <b>2</b>  |
| 1.1      | Bisection Method . . . . .                                   | 2         |
| 1.2      | Newton-Raphson Method . . . . .                              | 3         |
| 1.3      | Secant Method . . . . .                                      | 4         |
| 1.4      | Inverse Quadratic Interpolation (IQI) . . . . .              | 5         |
| <b>2</b> | <b>Interpolation</b>                                         | <b>6</b>  |
| 2.1      | Lagrange Interpolation . . . . .                             | 6         |
| 2.2      | Newton's Divided-Difference Interpolation . . . . .          | 7         |
| 2.3      | Natural Cubic Spline Interpolation . . . . .                 | 8         |
| <b>3</b> | <b>Numerical Integration</b>                                 | <b>10</b> |
| 3.1      | Trapezoidal Rule . . . . .                                   | 10        |
| 3.2      | Simpson's Rule . . . . .                                     | 11        |
| 3.3      | Midpoint Rule . . . . .                                      | 12        |
| 3.4      | Gaussian Quadrature . . . . .                                | 13        |
| <b>4</b> | <b>Linear Systems and Newton's Method for Systems</b>        | <b>14</b> |
| 4.1      | Gaussian Elimination . . . . .                               | 14        |
| 4.2      | LU Decomposition . . . . .                                   | 15        |
| 4.3      | PA = LU Decomposition (Partial Pivoting) . . . . .           | 16        |
| 4.4      | Newton's Method for Systems of Nonlinear Equations . . . . . | 17        |

# 1 Root-Finding Methods

## 1.1 Bisection Method

**Theory:** The Bisection method finds a root of  $f(x) = 0$  by repeatedly halving a bracketing interval  $[a, b]$  where  $f(a)f(b) < 0$ . It evaluates the midpoint  $c = (a + b)/2$  and replaces either  $a$  or  $b$  with  $c$  such that the root remains bracketed.

**Error Bounds & Convergence:** The error after  $n$  iterations is bounded by  $|x_n - r| \leq \frac{b-a}{2^{n+1}}$ . The convergence is **linear**.

```
1  import numpy as np
2
3  def f(x):
4      return (x)**3 + x - 1
5
6  a = 0.1
7  b = 1.0
8  tol = 10**(-8)
9
10 # Bisection Method implementation
11 if f(a)*f(b) > 0:
12     print('choose different points')
13 else:
14     while np.abs(b-a)/2 > tol:
15         c = 0.5*(a+b)
16         if f(a)*f(c) < 0:
17             b = c
18         else:
19             a = c
20
21     print('root = ', c)
22     print('f(c) = ', f(c))
```

## 1.2 Newton-Raphson Method

**Theory:** Uses the tangent line at the current guess to find the next guess. The iteration formula is  $x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$ .

**Error Bounds & Convergence:** The error scales quadratically near a simple root:  $\epsilon_{n+1} \approx \frac{f''(r)}{2f'(r)}\epsilon_n^2$ . Convergence is **quadratic**.

```
1  import numpy as np
2
3  def f(x):
4      return x**3 + x - 1
5
6  def fd(x):
7      return 3*x**2 + 1
8
9  x0 = 0.1
10 tol = 10**(-8)
11 val = 1
12
13 # Newton-Raphson implementation
14 if np.abs(f(x0)) < tol:
15     print('root = ', x0)
16 else:
17     while np.abs(val) > tol:
18         x = x0 - f(x0)/fd(x0)
19         val = (x - x0)
20         x0 = x
21
22     print('root = ', x)
```

### 1.3 Secant Method

**Theory:** Replaces the derivative in Newton's method with a finite-difference slope passing through the last two iterates.

**Error Bounds & Convergence:** The error scales as  $\epsilon_{n+1} \approx C\epsilon_n^\alpha$ , where  $\alpha = \frac{1+\sqrt{5}}{2} \approx 1.618$  (the golden ratio). Convergence is **superlinear**.

```
1  import numpy as np
2
3  def f(x):
4      return x**3 + x - 1
5
6  x0 = 0.1
7  x1 = 0.2
8  tol = 10**(-8)
9  val = 1
10
11  # Secant Method implementation
12  if np.abs(f(x0)) < tol:
13      print('root =', x0)
14  elif np.abs(f(x1)) < tol:
15      print('root =', x1)
16  else:
17      while np.abs(val) > tol:
18          fd = (f(x1) - f(x0))/(x1 - x0)
19          x = x1 - f(x1)/fd
20          val = (x - x1)
21          x0 = x1
22          x1 = x
23
24  print('root = ', x1)
```

## 1.4 Inverse Quadratic Interpolation (IQI)

**Theory:** Fits a sideways quadratic curve (where  $x = P(y)$ ) through the last three iterates.

$$P(y) = x_0 \frac{(y - y_1)(y - y_2)}{(y_0 - y_1)(y_0 - y_2)} + x_1 \frac{(y - y_0)(y - y_2)}{(y_1 - y_0)(y_1 - y_2)} + x_2 \frac{(y - y_0)(y - y_1)}{(y_2 - y_0)(y_2 - y_1)}$$

The next root guess is found by evaluating  $P(0)$ , which simplifies to:

$$x_3 = x_2 - \frac{r(r - q)(x_2 - x_1) + s(1 - r)(x_2 - x_0)}{(q - 1)(r - 1)(s - 1)}$$

**Error Bounds & Convergence:** The convergence is **superlinear**, with an order of  $\approx 1.84$ . It requires no derivatives but needs three initial guesses.

```
1  import numpy as np
2
3  def f(x):
4      return x**3 + x - 1
5
6  def iqi(f, x0, x1, x2, tol=1e-8, max_iter=100):
7      for i in range(max_iter):
8          y0, y1, y2 = f(x0), f(x1), f(x2)
9
10         # Check if we are close enough to zero
11         if abs(y2) < tol:
12             return x2
13
14         # These are the terms for the Lagrange interpolation formula (but sideways!)
15         # We calculate the next x guess by plugging y=0 into the sideways parabola
16         q = f(x0) / f(x1)
17         r = f(x2) / f(x1)
18         s = f(x2) / f(x0)
19
20         # Formula for the next guess (algebraically simplified)
21         x3 = x2 - (r * (r - q) * (x2 - x1) + s * (1 - r) * (x2 - x0)) / ((q - 1) * (r
22             - 1) * (s - 1))
23
24         # Shift everything over for the next loop
25         x0, x1, x2 = x1, x2, x3
26
27     return x2
28
29 # Example usage
30 root = iqi(f, 0.1, 0.15, 0.25)
31 print("Root is:", root)
```

## 2 Interpolation

### 2.1 Lagrange Interpolation

**Theory:** Builds the unique degree- $(n - 1)$  interpolating polynomial directly as a weighted sum of Lagrange basis polynomials  $L_i(x)$ .

$$P(x) = \sum_{i=0}^{n-1} y_i L_i(x) \quad \text{where} \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$

**Error Bounds:** The interpolation error is  $f(x) - P(x) = \frac{f^{(n)}(\xi)}{n!} \prod_{i=0}^{n-1} (x - x_i)$ .

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     return 1/(1 + 12*x**2)
6
7 # define continuous points for plotting
8 t = np.arange(-1, 1, 0.01)
9
10 # interpolation nodes
11 x = np.array([-1, -0.99, -0.97, -0.92, -0.8, -0.7, -0.5, -0.25, 0, 0.25, 0.5, 0.7,
12               0.8, 0.92, 0.97, 0.99, 1])
13
14 # init lagrange basis polynomial matrix
15 l = np.zeros((len(t), len(x)))
16
17 # compute basis polynomials automatically
18 for i in range(len(x)):
19     p = np.ones_like(t)
20     for j in range(len(x)):
21         if i != j:
22             p = p * (t - x[j]) / (x[i] - x[j])
23     l[:, i] = p
24
25 # lagrange interpolating polynomial for f(x)
26 L = np.zeros_like(t)
27 for i in range(len(x)):
28     L += f(x[i]) * l[:, i]
29
30 # plotting
31 plt.plot(t, f(t), label = 'f(t)')
32 plt.scatter(x, f(x), color = 'red', marker = '.', label = 'Interpolation nodes')
33 plt.plot(t, L, label = 'Lagrange Interpolating Polynomial')
34 plt.grid(True)
35 plt.title("Lagrange Interpolation [Un-equally spaced]")
36 plt.legend()
37 plt.show()
```

## 2.2 Newton's Divided-Difference Interpolation

**Theory:** Builds the same unique interpolating polynomial recursively using a divided-difference table. **Error Bounds:** Same error as Lagrange interpolation since the resulting polynomial is identical.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Fine grid for plotting the true function and interpolation
5 x = np.linspace(0, np.pi / 2, 100)
6 f = np.sin(x)
7
8 # Interpolation nodes and their function values
9 t = np.array([0, np.pi/6, 2*np.pi/6, 3*np.pi/6])
10 y = np.sin(t)
11
12 # --- 1. Calculate Newton Divided Difference Coefficients ---
13 n = len(t)
14 coef = np.zeros([n, n])
15 coef[:, 0] = y # first column is just the function values
16
17 for j in range(1, n):
18     for i in range(n - j):
19         coef[i+1][j] = (coef[i+1][j-1] - coef[i][j-1]) / (t[i+j] - t[i])
20
21 # The coefficients for the Newton polynomial are the top row of our table
22 c = coef[0, :]
23
24 # --- 2. Evaluate the Newton Interpolating Polynomial ---
25 P_x = np.zeros_like(x)
26 for i in range(len(x)):
27     val = c[0]
28     prod = 1.0
29     for j in range(1, n):
30         prod *= (x[i] - t[j-1])
31         val += c[j] * prod
32     P_x[i] = val
33
34 # --- 3. Plotting ---
35 plt.plot(x, f, label='True function: sin(x)')
36 plt.scatter(t, y, color='red', marker='x', label='Interpolation nodes')
37 plt.plot(x, P_x, label='Newton Interpolating Polynomial', linestyle='--')
38 plt.legend()
39 plt.title("Newton Divided Difference Interpolation")
40 plt.grid(True)
41 plt.show()
```

## 2.3 Natural Cubic Spline Interpolation

**Theory:** Fits a piecewise-cubic interpolant ensuring continuous value, slope, and curvature at every interior node. The "Natural" boundary condition specifies that the curvature at the endpoints is zero ( $S''(x_1) = S''(x_n) = 0$ ).

**Error Bounds:** The error bound for cubic splines is  $|f(x) - S(x)| \leq \frac{5}{384} \max |f^{(4)}(x)| h^4$ .

```

1 import numpy as np
2 from matplotlib import pyplot as plt
3
4 def compute_natural_cubic_spline(x, y):
5     """
6     Computes coefficients for the piecewise cubic polynomials:
7      $S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$ 
8     """
9     x = np.array(x, dtype=float)
10    y = np.array(y, dtype=float)
11    n = len(x)
12
13    # 1. Widths of the intervals
14    h = np.diff(x)
15
16    # 2. Set up the linear system system:  $A * c = B$ 
17    A = np.zeros((n, n))
18    B = np.zeros(n)
19
20    # Natural boundary conditions: second derivative at endpoints is 0
21    A[0, 0] = 1.0
22    A[n-1, n-1] = 1.0
23
24    # Fill the tridiagonal matrix based on continuity conditions
25    for i in range(1, n-1):
26        A[i, i-1] = h[i-1]
27        A[i, i] = 2 * (h[i-1] + h[i])
28        A[i, i+1] = h[i]
29        B[i] = 3 * ((y[i+1] - y[i]) / h[i] - (y[i] - y[i-1]) / h[i-1])
30
31    # 3. Solve for 'c' coefficients
32    c = np.linalg.solve(A, B)
33
34    # 4. Derive 'a', 'b', and 'd' coefficients from 'c'
35    a = y[:-1]
36    b = np.zeros(n - 1)
37    d = np.zeros(n - 1)
38
39    for i in range(n - 1):
40        b[i] = (y[i+1] - y[i]) / h[i] - h[i] * (2 * c[i] + c[i+1]) / 3
41        d[i] = (c[i+1] - c[i]) / (3 * h[i])
42
43    return a, b, c[:-1], d
44
45 def evaluate_spline(x_query, x_data, a, b, c, d):
46     """Evaluates the spline at given query coordinates."""
47     y_query = []
48     for x_val in x_query:
49         # Find which interval the query point falls into
50         if x_val <= x_data[0]:
51             i = 0
52         elif x_val >= x_data[-2]:
53             i = len(x_data) - 2
54         else:
55             # Binary search to locate interval index
56             i = np.searchsorted(x_data, x_val) - 1
57
58         dx = x_val - x_data[i]
59         val = a[i] + b[i]*dx + c[i]*(dx**2) + d[i]*(dx**3)
60         y_query.append(val)
61
62     return np.array(y_query)
63
64 # --- Example Execution & Plotting ---
65 x_points = np.array([0.0, 1.0, 2.0, 4.0])
66 y_points = np.array([1.0, 3.0, 2.0, 5.0])
67
68 a, b, c, d = compute_natural_cubic_spline(x_points, y_points)
69 x_dense = np.linspace(-1, 5, 200)
70 y_dense = evaluate_spline(x_dense, x_points, a, b, c, d)
71
72 plt.figure(figsize=(8, 5))
73 plt.scatter(x_points, y_points, color='red', label='Data Points', zorder=5, marker='.',
74            ')
75 plt.plot(x_dense, y_dense, color='blue', label='Custom Cubic Spline')

```

## 3 Numerical Integration

### 3.1 Trapezoidal Rule

**Theory:** Approximates the integral by treating the area under the curve as a series of trapezoids.

$$\text{Area} = \frac{b-a}{2}(f(a) + f(b))$$

$$\int_a^b f(x)dx \approx \frac{h}{2} \left[ f(x_0) + 2 \sum_{i=1}^{m-1} f(x_i) + f(x_m) \right]$$

**Error Bounds:**

- Local Error (Single Panel):  $-\frac{h^3}{12}f''(\xi)$
- Global Error (Composite):  $-\frac{b-a}{12}h^2f''(\xi)$

```
1  import numpy as np
2
3  def f(x):
4      return x**2
5
6  def trapezoidal_single(f, a, b):
7      h = b - a
8      area = h * (f(a) + f(b)) / 2
9      return area
10
11 def trapezoidal_composite(f, a, b, num_panels):
12     h = (b - a) / num_panels
13     x = np.linspace(a, b, num_panels + 1)
14
15     # Area = h/2 * [f(x_first) + 2*f(x_middles) + f(x_last)]
16     # The middle points get counted twice because they touch two trapezoids
17     area = (h / 2) * (f(x[0]) + 2 * np.sum(f(x[1:-1])) + f(x[-1]))
18     return area
19
20 # Example usage
21 print("Single Trapezoid Area:", trapezoidal_single(f, 0, 1))
22 print("Composite Trapezoid Area (100 panels):", trapezoidal_composite(f, 0, 1, 100))
```

## 3.2 Simpson's Rule

**Theory:** Approximates the integral by fitting a parabola through sets of three points (the endpoints and the midpoint).

$$\text{Area} = \frac{h}{3}(f(a) + 4f(m) + f(b))$$
$$\int_a^b f(x)dx \approx \frac{h}{3} \left[ f(x_0) + 4 \sum_{\text{odd } i} f(x_i) + 2 \sum_{\text{even } i} f(x_i) + f(x_m) \right]$$

**Error Bounds:**

- Local Error (Single Panel):  $-\frac{h^5}{90}f^{(4)}(\xi)$
- Global Error (Composite):  $-\frac{b-a}{180}h^4f^{(4)}(\xi)$

```
1 import numpy as np
2
3 def f(x):
4     return x**2
5
6 def simpson_single(f, a, b):
7     # A single application requires 3 points: the ends and the exact middle
8     midpoint = (a + b) / 2
9     h = (b - a) / 2
10
11     # Formula uses weights: 1, 4, 1
12     area = (h / 3) * (f(a) + 4*f(midpoint) + f(b))
13     return area
14
15 def simpson_composite(f, a, b, num_panels):
16     # Simpson's rule needs an EVEN number of panels
17     if num_panels % 2 != 0:
18         raise ValueError("Number of panels must be even for Simpson's Rule.")
19
20     h = (b - a) / num_panels
21     x = np.linspace(a, b, num_panels + 1)
22
23     # Weights follow a pattern: 1, 4, 2, 4, 2, ... 4, 1
24     y = f(x)
25     area = (h / 3) * (y[0] + 4 * np.sum(y[1:-1:2]) + 2 * np.sum(y[2:-1:2]) + y[-1])
26     return area
27
28 # Example usage
29 print("Single Simpson Area:", simpson_single(f, 0, 1))
30 print("Composite Simpson Area (100 panels):", simpson_composite(f, 0, 1, 100))
```

### 3.3 Midpoint Rule

**Theory:** Approximates the area with flat rectangles evaluated exactly at the midpoint of each panel. It avoids evaluating the function at the endpoints.

$$\text{Area} = h \cdot f(m)$$

$$\int_a^b f(x)dx \approx h \sum_{i=1}^n f(m_i)$$

#### Error Bounds:

- Local Error (Single Panel):  $\frac{h^3}{24}f''(\xi)$
- Global Error (Composite):  $\frac{b-a}{24}h^2f''(\xi)$

```
1  import numpy as np
2
3  def f(x):
4      return x**2
5
6  def midpoint_single(f, a, b):
7      midpoint = (a + b) / 2
8      h = b - a
9      area = h * f(midpoint)
10     return area
11
12 def midpoint_composite(f, a, b, num_panels):
13     h = (b - a) / num_panels
14
15     # Get the exact midpoints of every single panel
16     midpoints = np.linspace(a + h/2, b - h/2, num_panels)
17
18     # We just sum the height of all the rectangles and multiply by width
19     area = h * np.sum(f(midpoints))
20     return area
21
22 # Example usage
23 print("Single Midpoint Area:", midpoint_single(f, 0, 1))
24 print("Composite Midpoint Area (100 panels):", midpoint_composite(f, 0, 1, 100))
```

### 3.4 Gaussian Quadrature

**Theory:** Optimally chooses the nodes ( $t_i$ ) and weights ( $c_i$ ) to maximize the exactness of the integral.

$$\int_{-1}^1 f(t)dt \approx \sum_{i=1}^n c_i f(t_i)$$
$$\int_a^b f(x)dx \approx \frac{b-a}{2} \sum_{i=1}^n c_i f\left(\frac{b-a}{2}t_i + \frac{b+a}{2}\right)$$

**Error Bounds:** An  $n$ -point Gaussian Quadrature rule is completely exact for all polynomials up to degree  $2n - 1$ . The error for general functions is proportional to  $f^{(2n)}(\xi)$ .

```
1  import numpy as np
2
3  def f(x):
4      return np.exp(-x**2 / 2)
5
6  def gaussian_quadrature(f, a, b, n):
7      # NumPy has a built-in function to generate the perfect optimal points (t) and
8      their weights (c)
9      t, c = np.polynomial.legendre.leggauss(n)
10
11     # The points it gives are from -1 to 1. We have to map them to our actual
12     interval [a, b]
13     mapped_x = 0.5 * (b - a) * t + 0.5 * (b + a)
14
15     # Multiply the weights by our function evaluated at those perfect points
16     area = 0.5 * (b - a) * np.sum(c * f(mapped_x))
17     return area
18
19 # Example usage (Notice how accurate it is with just 4 points!)
20 print("Gaussian Quadrature (4 points):", gaussian_quadrature(f, -1, 1, 4))
```

## 4 Linear Systems and Newton's Method for Systems

### 4.1 Gaussian Elimination

**Theory:** Transforms a system  $A\mathbf{x} = \mathbf{b}$  into an upper-triangular matrix via forward elimination, and then solves for  $\mathbf{x}$  using backward substitution.

**Error Bounds & Stability:** Vulnerable to round-off error if pivots are close to zero. The residual error heavily depends on the condition number of the matrix  $A$ .

```
1  import numpy as np
2
3  def gauss_elimination(A, b):
4      n = len(b)
5      # Forward elimination
6      for i in range(n):
7          for j in range(i+1, n):
8              factor = A[j, i] / A[i, i]
9              A[j] -= factor * A[i]
10             b[j] -= factor * b[i]
11
12     # Back substitution
13     x = np.zeros(n)
14     for i in range(n-1, -1, -1):
15         known_chunk = np.dot(A[i, i+1:], x[i+1:])
16         x[i] = (b[i] - known_chunk) / A[i, i]
17
18     return x
19
20 # Example 3x3 system
21 A = np.array([[3.0, 2.0, -4.0], [2.0, 3.0, 3.0], [5.0, -3.0, 1.0]])
22 b = np.array([3.0, 15.0, 14.0])
23
24 x = gauss_elimination(A.copy(), b.copy())
25 print("Solution x:", x)
```

## 4.2 LU Decomposition

**Theory:** Factorizes  $A = LU$  (Lower and Upper triangular matrices) to solve  $A\mathbf{x} = \mathbf{b}$  efficiently for multiple  $\mathbf{b}$  vectors. We solve  $L\mathbf{y} = \mathbf{b}$  followed by  $U\mathbf{x} = \mathbf{y}$ .

```
1  import numpy as np
2
3  def lu_decomposition(A):
4      n = A.shape[0]
5      L = np.eye(n)
6      U = A.copy()
7
8      for i in range(n):
9          for j in range(i+1, n):
10             factor = U[j, i] / U[i, i]
11             U[j] -= factor * U[i]
12             L[j, i] = factor
13
14     return L, U
15
16 def solve_lu(L, U, b):
17     # 1. Forward substitution (Solve L y = b)
18     y = np.linalg.solve(L, b)
19
20     # 2. Backward substitution (Solve U x = y)
21     x = np.linalg.solve(U, y)
22
23     return x
24
25 A = np.array([[3.0, 2.0, -4.0], [2.0, 3.0, 3.0], [5.0, -3.0, 1.0]])
26 b = np.array([3.0, 15.0, 14.0])
27
28 # Factorize once
29 L, U = lu_decomposition(A)
30
31 # Solve for x
32 x = solve_lu(L, U, b)
33
34 print("L matrix:\n", L)
35 print("\nU matrix:\n", U)
36 print("\nVerify L @ U == A:\n", np.allclose(L @ U, A))
37 print("\nSolution x:", x)
```

### 4.3 PA = LU Decomposition (Partial Pivoting)

**Theory:** Implements partial pivoting by swapping rows to ensure the largest possible absolute pivot is used. This prevents division by zero and maintains extreme numerical stability.

```
1  import numpy as np
2
3  def pa_lu_decomposition(A):
4      n = A.shape[0]
5      L = np.eye(n)
6      U = A.copy()
7      P = np.eye(n)
8
9      for i in range(n):
10         # Partial Pivoting: find the row with the largest absolute value in the
11         current column
12         max_idx = np.argmax(np.abs(U[i:, i])) + i
13
14         # Swap rows in U, P, and the previously calculated parts of L
15         if i != max_idx:
16             # Swap in U
17             U[i], U[max_idx] = U[max_idx].copy(), U[i].copy()
18             # Swap in P
19             P[i], P[max_idx] = P[max_idx].copy(), P[i].copy()
20             # Swap in L (only the columns up to i)
21             if i > 0:
22                 L[i, :i], L[max_idx, :i] = L[max_idx, :i].copy(), L[i, :i].copy()
23
24         # Standard LU elimination steps
25         for j in range(i+1, n):
26             factor = U[j, i] / U[i, i]
27             L[j, i] = factor
28             U[j] -= factor * U[i]
29
30     return P, L, U
31
32 def solve_pa_lu(P, L, U, b):
33     # 1. Apply permutation to b (We are solving L U x = P b)
34     Pb = P @ b
35
36     # 2. Forward substitution (Solve L y = Pb)
37     y = np.linalg.solve(L, Pb)
38
39     # 3. Backward substitution (Solve U x = y)
40     x = np.linalg.solve(U, y)
41
42     return x
43
44 # Example using a matrix that would fail standard LU (0 on diagonal)
45 A = np.array([[0.0, 2.0, 3.0], [1.0, -1.0, 1.0], [-1.0, 1.0, 0.0]])
46 b = np.array([4.0, 3.0, 1.0])
47
48 P, L, U = pa_lu_decomposition(A)
49 x = solve_pa_lu(P, L, U, b)
50
51 print("Verify P @ A == L @ U:\n", np.allclose(P @ A, L @ U))
52 print("\nSolution x:", x)
```

## 4.4 Newton's Method for Systems of Nonlinear Equations

**Theory:** Solves  $\mathbf{F}(\mathbf{x}) = \mathbf{0}$  for multi-variable systems. We repeatedly compute the Jacobian matrix  $\mathbf{J}(\mathbf{x})$ , find its inverse  $\mathbf{J}^{-1}$ , and update our guess using the standard inverse formula:

$$\mathbf{x}_{i+1} = \mathbf{x}_i - [\mathbf{J}(\mathbf{x}_i)]^{-1}\mathbf{F}(\mathbf{x}_i)$$

**Error Bounds & Convergence:** Like single-variable Newton's Method, convergence is **quadratic** near the solution, assuming the Jacobian is non-singular.

```
1  import numpy as np
2
3  # We have two equations we want to solve simultaneously:
4  # 1) x1 - x0^3 = 0
5  # 2) x0^2 + x1^2 - 1 = 0
6  def F(x):
7      return np.array([
8          x[1] - x[0]**3,
9          x[0]**2 + x[1]**2 - 1
10     ])
11
12 # The Jacobian is a matrix of the partial derivatives of those equations
13 def Jacobian(x):
14     return np.array([
15         [-3 * x[0]**2, 1],
16         [2 * x[0], 2 * x[1]]
17     ])
18
19 def newton_system(F, Jacobian, initial_guess, tol=1e-6, max_iter=100):
20     x = np.array(initial_guess, dtype=float)
21
22     for i in range(max_iter):
23         J = Jacobian(x)
24         fx = F(x)
25
26         # Find the inverse of the Jacobian
27         J_inv = np.linalg.inv(J)
28
29         # Calculate the step: - J_inv * F(x)
30         step = -(J_inv @ fx)
31
32         # Update x: x = x + step
33         x = x + step
34
35         # If the step was tiny, we have converged!
36         if np.linalg.norm(step) < tol:
37             print(f"Converged in {i+1} iterations.")
38             return x
39
40     return x
41
42 # Example usage
43 initial_guess = [1.0, 2.0]
44 solution = newton_system(F, Jacobian, initial_guess)
45 print("Solution:", solution)
```